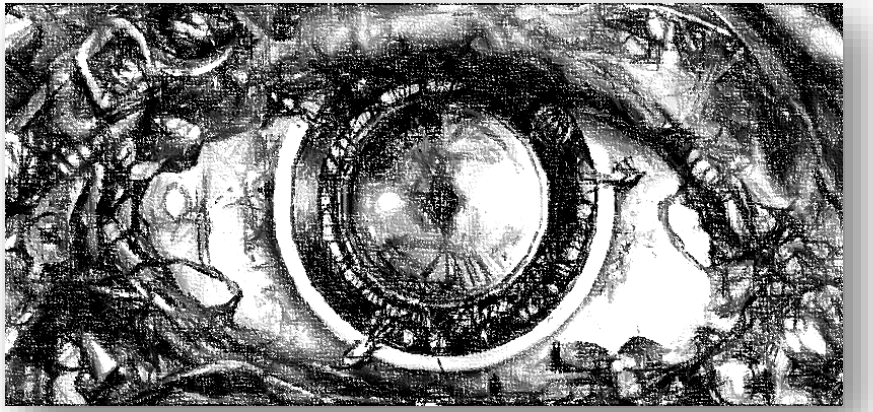
Image Classification



Departement of Informatics
Faculty of Intelligent Electrical and Informatics
Institut Teknologi Sepuluh Nopember
Surabaya, 2026

COMPUTER VISION for ANALYZING VISUAL INFORMATION

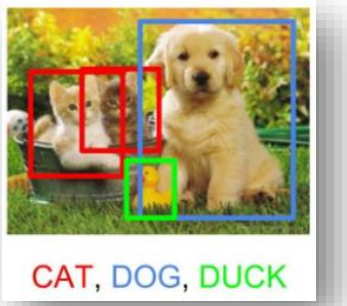
- Text



Computer Vision enables the computer to imitate human abilities in seeing and understanding the environment to complete certain tasks based on visual information captured by the eye.



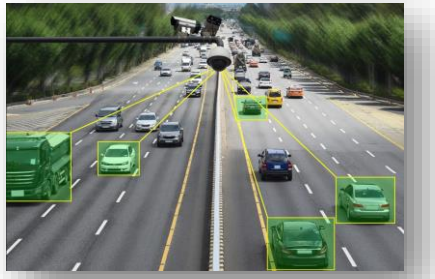
Image Classification



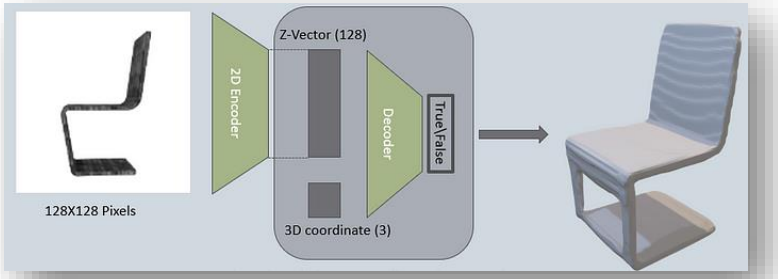
Object Detection



Instance Segmentation



Tracking

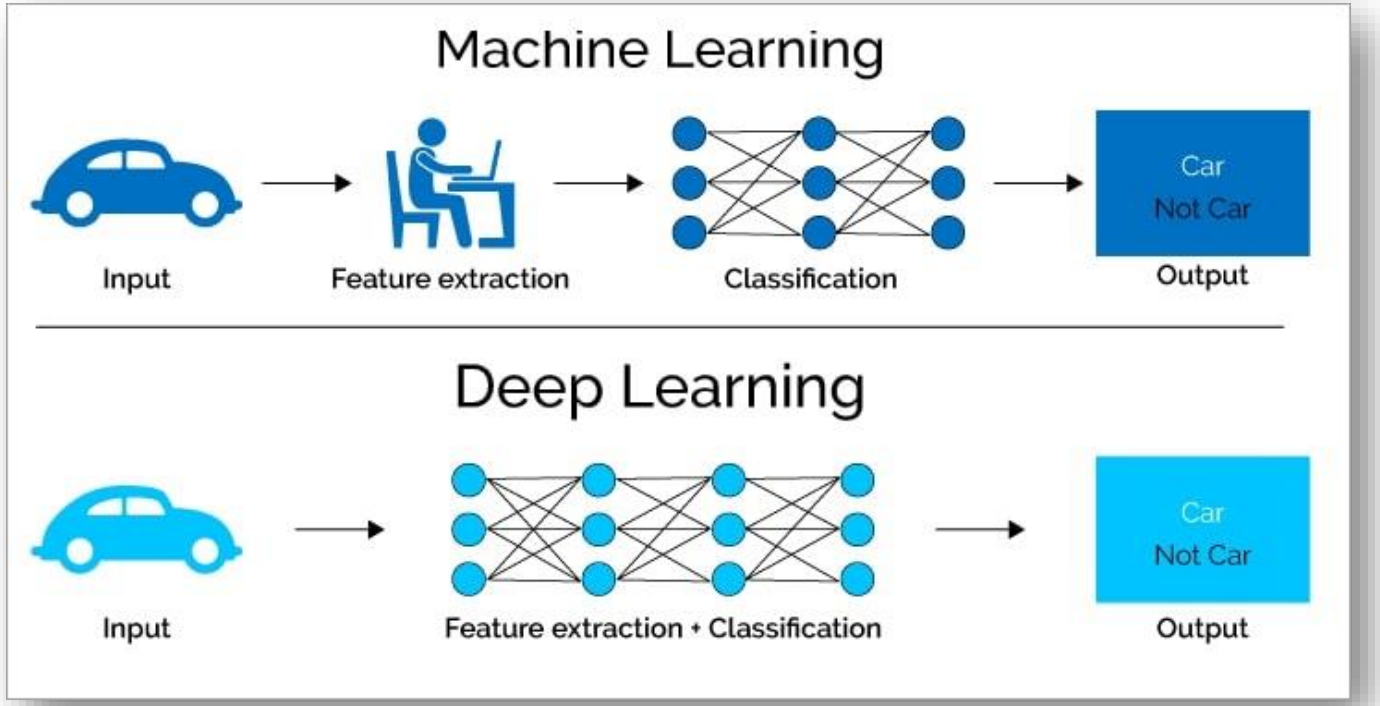


3D Reconstruction



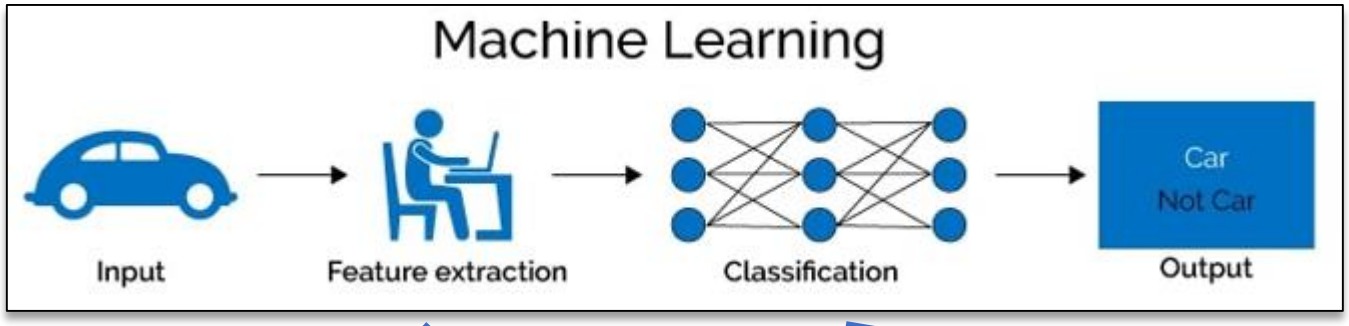
Image Captioning

IMAGE CLASSIFICATION



- Machine learning and deep learning are two common approaches in computer vision.
- The machine learning approach use a (hand-crafted/manual) feature extraction process to extract the important information of an image, and use the information or features to train a classification model.
- The Deep Learning approach uses a deep ANN architecture, which is trained end-to-end with a large dataset and is simultaneously used for feature extraction and classification.

MACHINE LEARNING APPROACH



Preprocessing

- De-noising
- Enhancement
- Blurring
- ….

Structured Data

Feature Extraction Methods

- Gray-level Co-occurrence Matrix
- Local Binary Pattern
- Wavelet
- Histogram of Oriented Gradients
- Bag of Visual Words
- Discrete Cosinous Transfrom
- ….

Optimization Methods for feature selection/weighting or/and parameter tuning

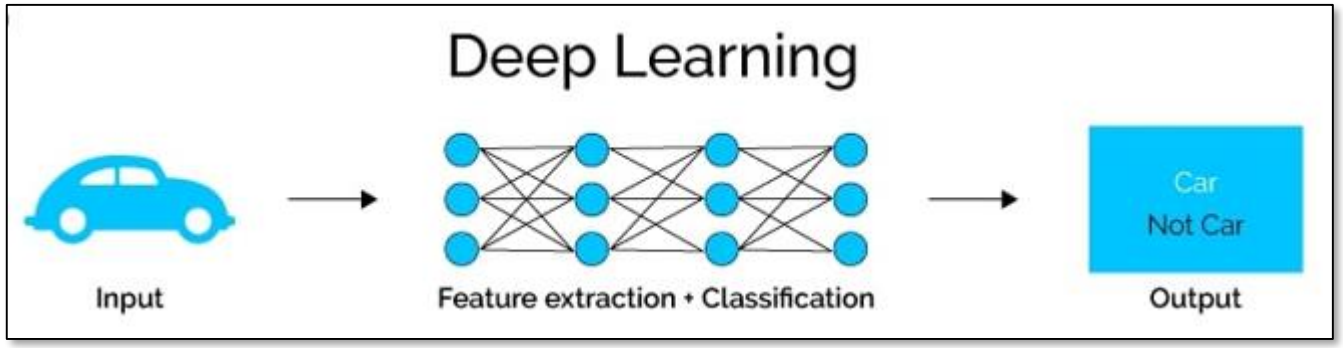
- Genetic Algorithm
- Particle Swarm Optimization
- Ant Colony Optimization
- …

Classification Methods

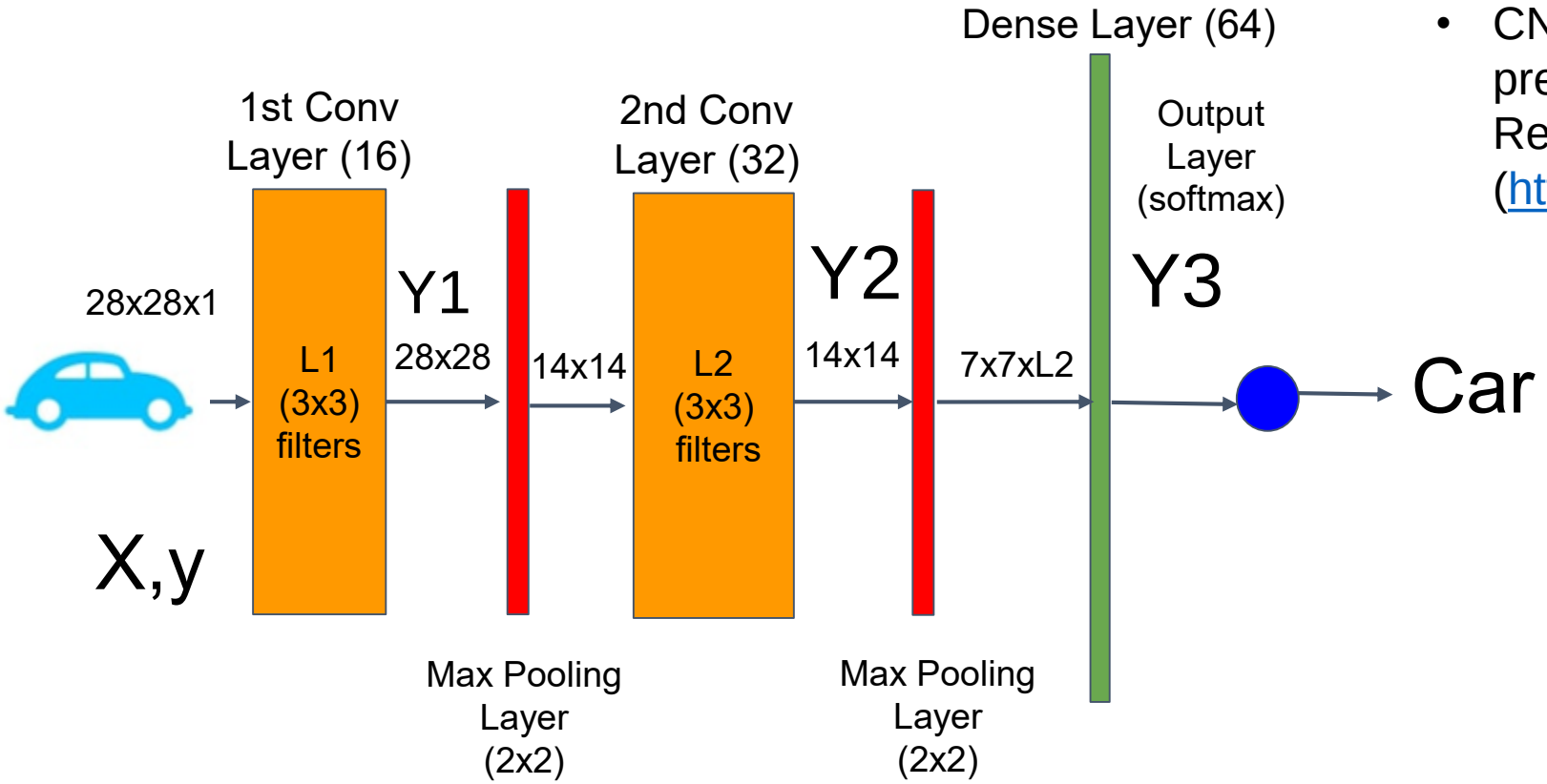
- k-Nearest Neighbour
- Naïve Bayes
- Decision Tree
- Support Vector Machine
- Artificial Neural Network
- …

Feature Fusion Ensemble Classification …

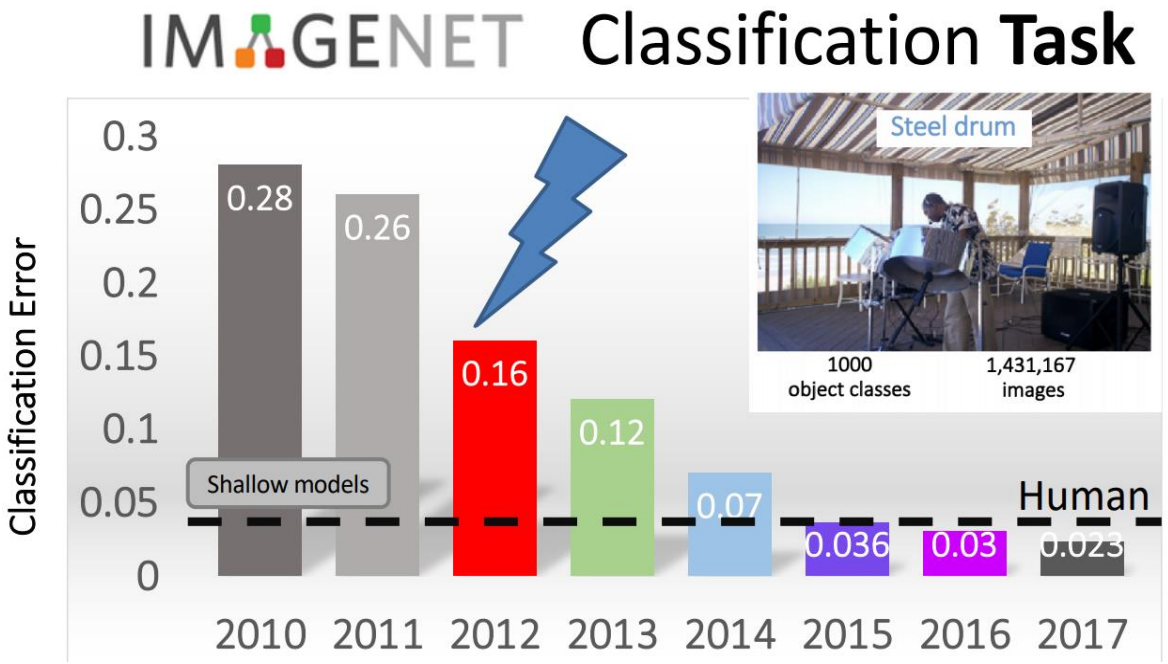
DEEP LEARNING APPROACH



- The Deep Learning approach adopt a ANN with deep layers (Convolutional Neural Network/CNN) , which is trained end-to-end and is simultaneously used for feature extraction and classification.
- CNN is designed to recognize visual patterns directly from pixel images with minimal preprocessing and can learn spatial hierarchies local patterns.
- CNN was invented by Yann LeCun, previously was the Director of AI Research, Facebook (http://yann.lecun.com/).



IMAGENET Large Scale Visual Recognition Challenge



ILSVRC 2010
Lin et. al. from NEC with their article “Large-scale image classification: Fast feature extraction and SVM training”

ILSVRC 2011
Perronnin et. al. from XEROC with their article “High-Dimensional Signature Compression for Large-Scale Image Classification”

ILSVRC 2012
AlexNet (SuperVision)
Alex Krizhevsky, et al. from the University of Toronto in their 2012 paper titled “ImageNet Classification with Deep Convolutional Neural Networks” developed a convolutional neural network that achieved top results on the ILSVRC-2012 image classification tasks.

ILSVRC 2013
ZFNet (Clarifai)
Matthew Zeiler and Rob Fergus propose a variation of AlexNet generally referred to as ZFNet in their 2013 paper titled “Visualizing and Understanding Convolutional Networks,” a variation of which won the ILSVRC-2013 image classification task.

ILSVRC 2014
Inception (GoogLeNet)
Christian Szegedy, et al. from Google achieved top results for object detection with their GoogLeNet model that made use of the inception module and architecture. This approach was described in their 2014 paper titled “Going Deeper with Convolutions.”
VGG
Karen Simonyan and Andrew Zisserman from the Oxford Vision Geometry Group (VGG) achieved top results for image classification and localization with their VGG model. Their approach is described in their 2015 paper titled “Very Deep Convolutional Networks for Large-Scale Image Recognition.”
ILSVRC 2015
ResNet (MSRA)
Kaiming He, et al. from Microsoft Research achieved top results for object detection and object detection with localization tasks with their Residual Network or ResNet described in their 2015 paper titled “Deep Residual Learning for Image Recognition.”

- On many computer vision tasks, performance of the deep learning outperforms the machine learning (with separated hand-crafted feature extraction).
- However, machine learning (conventional) approaches are still often used, especially when the amount of data is limited, feature extraction precision is critical, or when a more structured understanding of the processing flow is required.

Challenges

Some of the key challenges in applying deep learning to computer vision include:

- **Data Quality and Quantity:**

Requires large amounts of labeled data for training

- **Computational Resources:**

Need significant computational power

- **Overfitting:**

Deep learning models are prone to overfitting, especially when the training dataset is small.

- **Interpretability:**

Challenging to interpret the reasoning behind their predictions

- **Transfer Learning:**

Requires fine tuning with careful consideration to domain differences and dataset characteristics.

- **Robustness and Adversarial Attacks:**

Deep learning models can be sensitive to small changes in input data.

…

Introduction to CNN

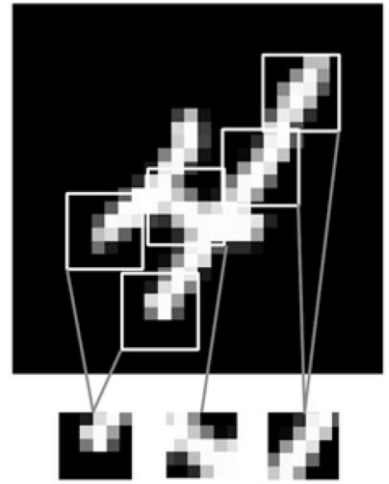
Introduction to CNN

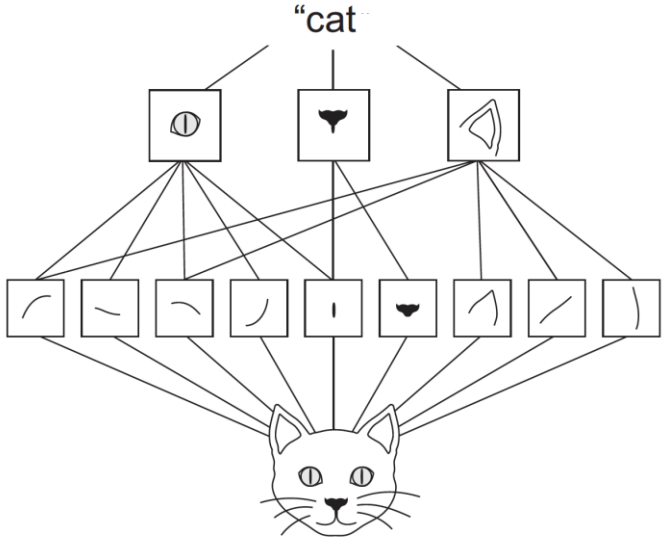
- CNN stands for Convolutional Neural Network. CNN is designed to recognize visual patterns directly from pixel images with minimal preprocessing.
- CNN was invented by Yann LeCun, currently the Director of AI Research, Facebook (http://yann.lecun.com/)
- LeNet-5 is the latest convolutional network designed for handwritten and machine-printed character recognition (http://yann.lecun.com/exdb/lenet/index.html)



CNN vs Feedforward NN

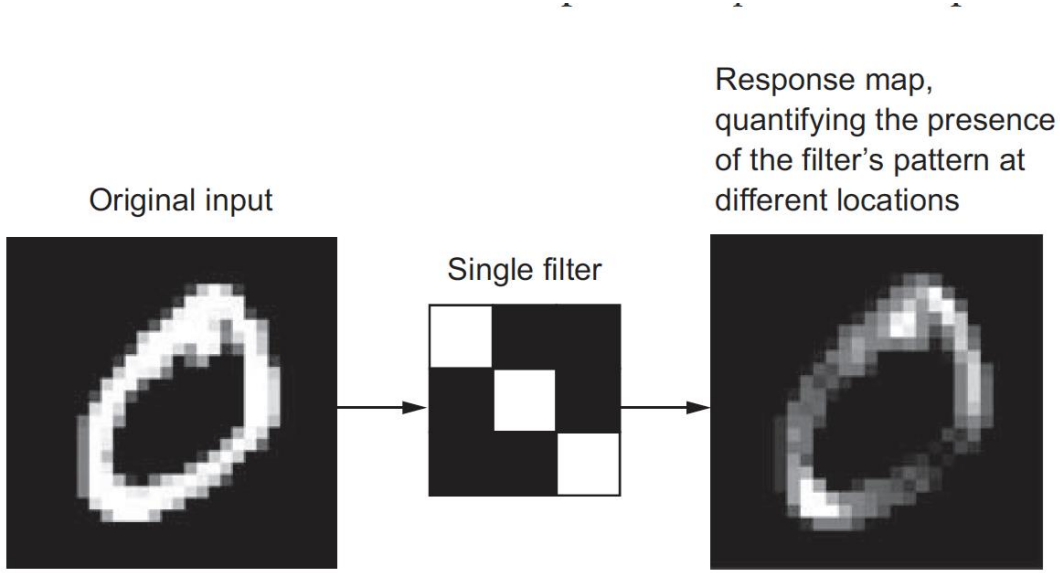
- A feedforward NN learn the global patterns
- A Convolutional Neural Network (CNN) learn local patterns.
- The key characteristic of CNN are:
  - The patterns they learn are translation invariant.
  - They can learn spatial hierarchies of patterns





How CNN Work?

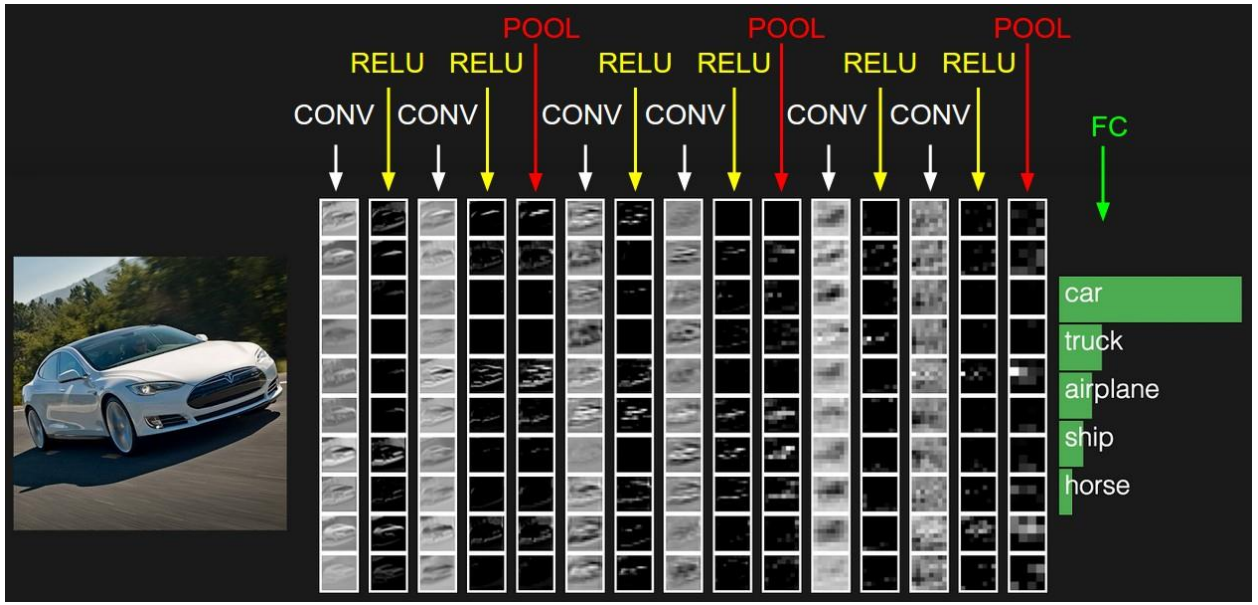
- Convolutions operate over 3D tensors (heigh, width, depth/channel)
- For MNIST , the first convolution layer with 32 filters takes a feature map of size (28, 28, 1) and computes 32 filters over its input
- The outputs is a feature map of size (26, 26, 32) response map



ConvNET Architecture

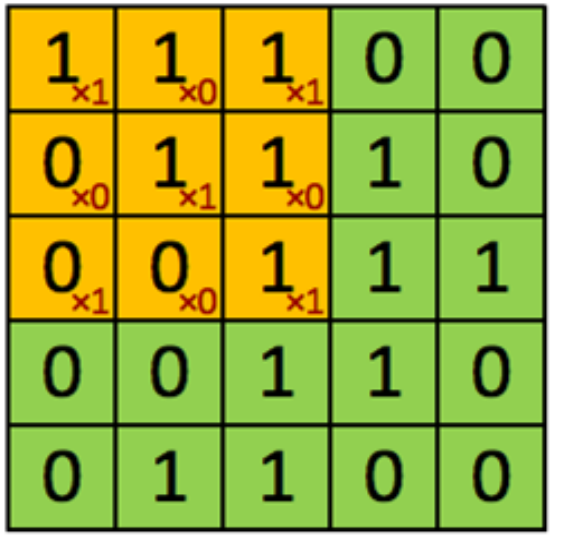
ConvNET architecture consists the follow layers

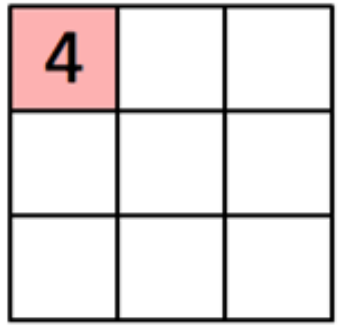
- Convolutional layer
- Activation layer
- Max Pooling layer
- Dropout layer
- Fully Connected layer



Convolution

- Convolution is the process of sliding a filter across the image
- The filter act as a feature detector





Vertical Edge Detection Filter

Image

30	30	30	0	0	0
30	30	30	0	0	0
30	30	30	0	0	0
30	30	30	0	0	0
30	30	30	0	0	0
30	30	30	0	0	0

*

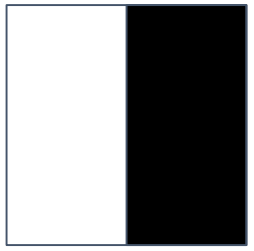
Filter

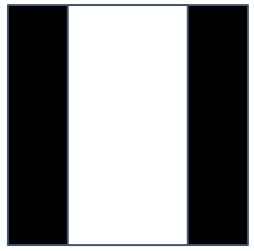
1	0	-1
1	0	-1
1	0	-1

=

Convoluted Feature

0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0





Output Image Size

The output image size can be computed by this formula

(N-F)/stride + 1 Eg (5-3)/1+1 = 3

N: input size (7-3)/1+1 = 5

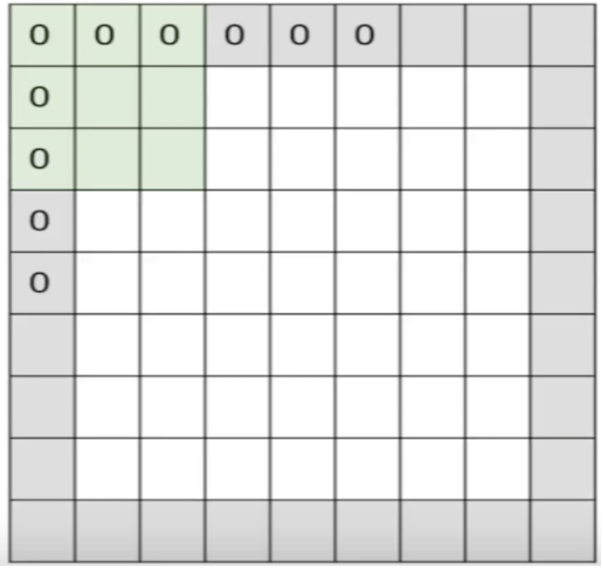
F: filter size

S: stride

Stride is the step taken when sliding the filter

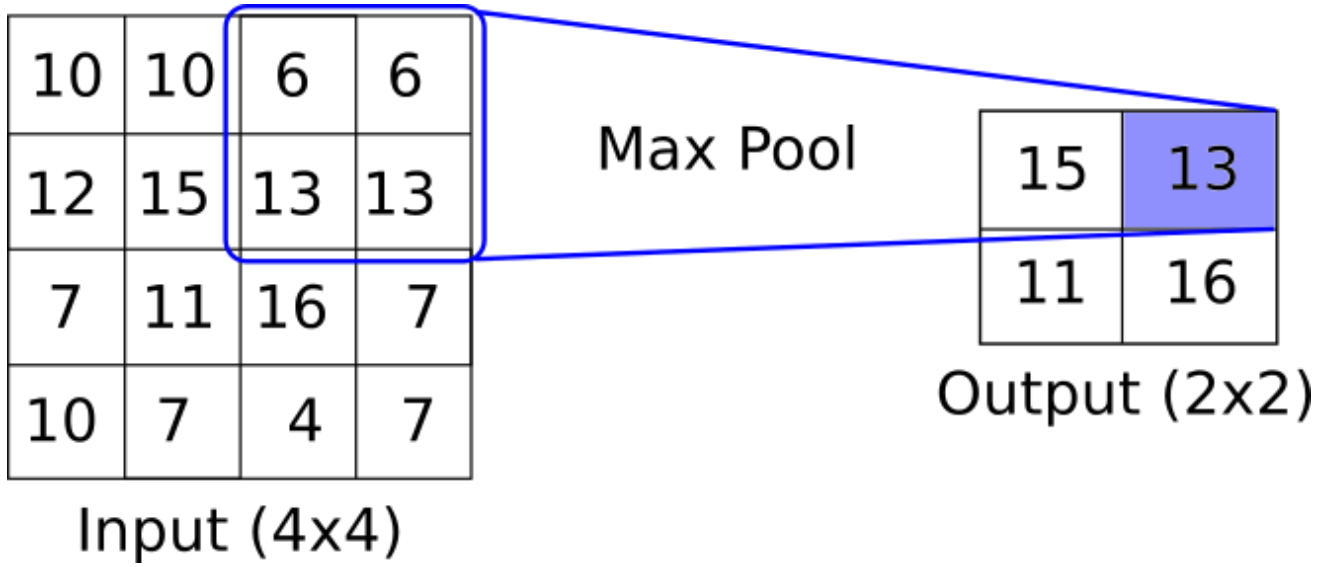
Padding

- Zero padding is used to preserve the output image size and reduce edge information loss.
- There are two types of padding
  - Valid padding - no padding
  - Same padding - keep the same output image size



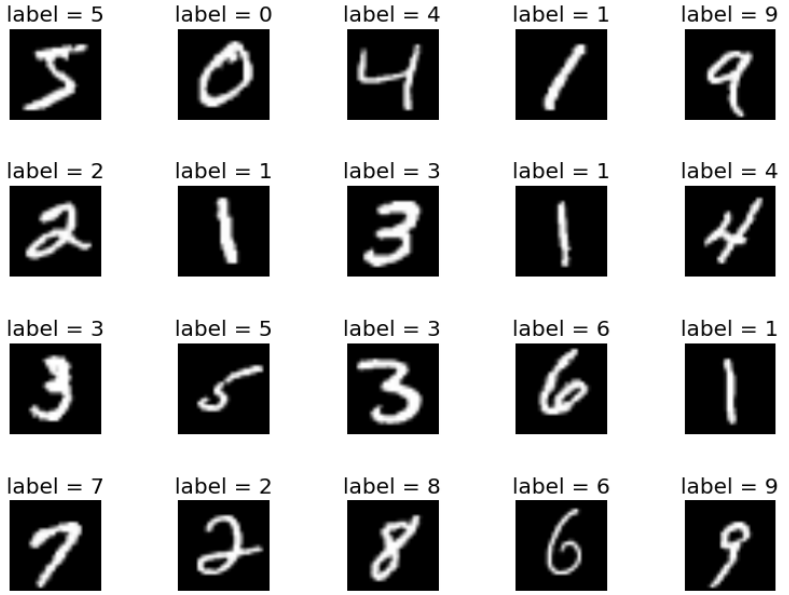
Pooling Layer

- Similar to stride, the purpose of a pooling layer is to downsample the output size. and extract the salient features.
- Maximum or average pooling layers



MNIST Handwritten Digits

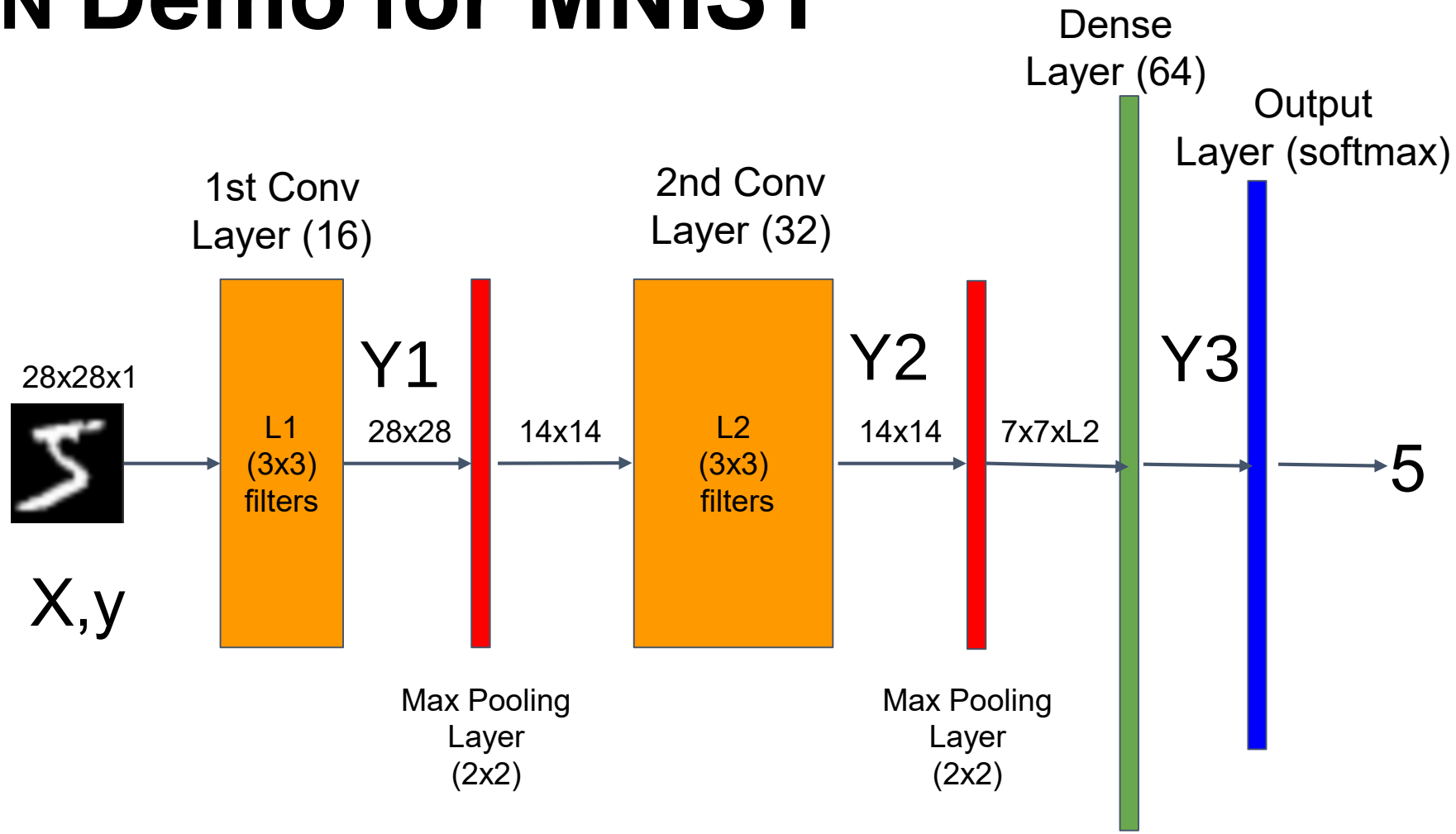
- The MNIST data is split into three parts:
  - 55,000 training data
  - 10,000 test data
  - 5,000 validation data
- Each image is 28 pixels by 28 pixels and serialize to a 1D vector of 784 numbers
- The label is one hot encoded.



One Hot Encoding for Digits

0	[1 0 0 0 0 0 0 0 0 0]
1	[0 1 0 0 0 0 0 0 0 0]
2	[0 0 1 0 0 0 0 0 0 0]
3	[0 0 0 1 0 0 0 0 0 0]
4	[0 0 0 0 1 0 0 0 0 0]
5	[0 0 0 0 0 1 0 0 0 0]
6	[0 0 0 0 0 0 1 0 0 0]
7	[0 0 0 0 0 0 0 1 0 0]
8	[0 0 0 0 0 0 0 0 1 0]
9	[0 0 0 0 0 0 0 0 0 1]

CNN Demo for MNIST



CNN with Keras

```
model = Sequential()
model.add(Conv2D(16,(3,3),activation='relu',
          input_shape=(28,28,1),padding='same'))
model.add(MaxPooling2D(2,2))
model.add(Conv2D(32,(3,3),activation='relu',
                    padding='same'))
model.add(MaxPooling2D(2,2))
model.add(Flatten())
model.add(Dense(64,activation='relu'))
model.add(Dense(10,activation='softmax'))
```

Cats and Dogs Small Dataset

- Access the cats and dogs data below
- https://drive.google.com/drive/folders/1Z85ibJN09AGXg3AelzwyEDIqLZxt6D5o?usp=sharing
- Download to your google drive
- There are 3 folders - train, test,validate
- Each one has 500 cats and dogs images

train_dir = '/content/gdrive/My Drive/data/cats-dogs/train/'

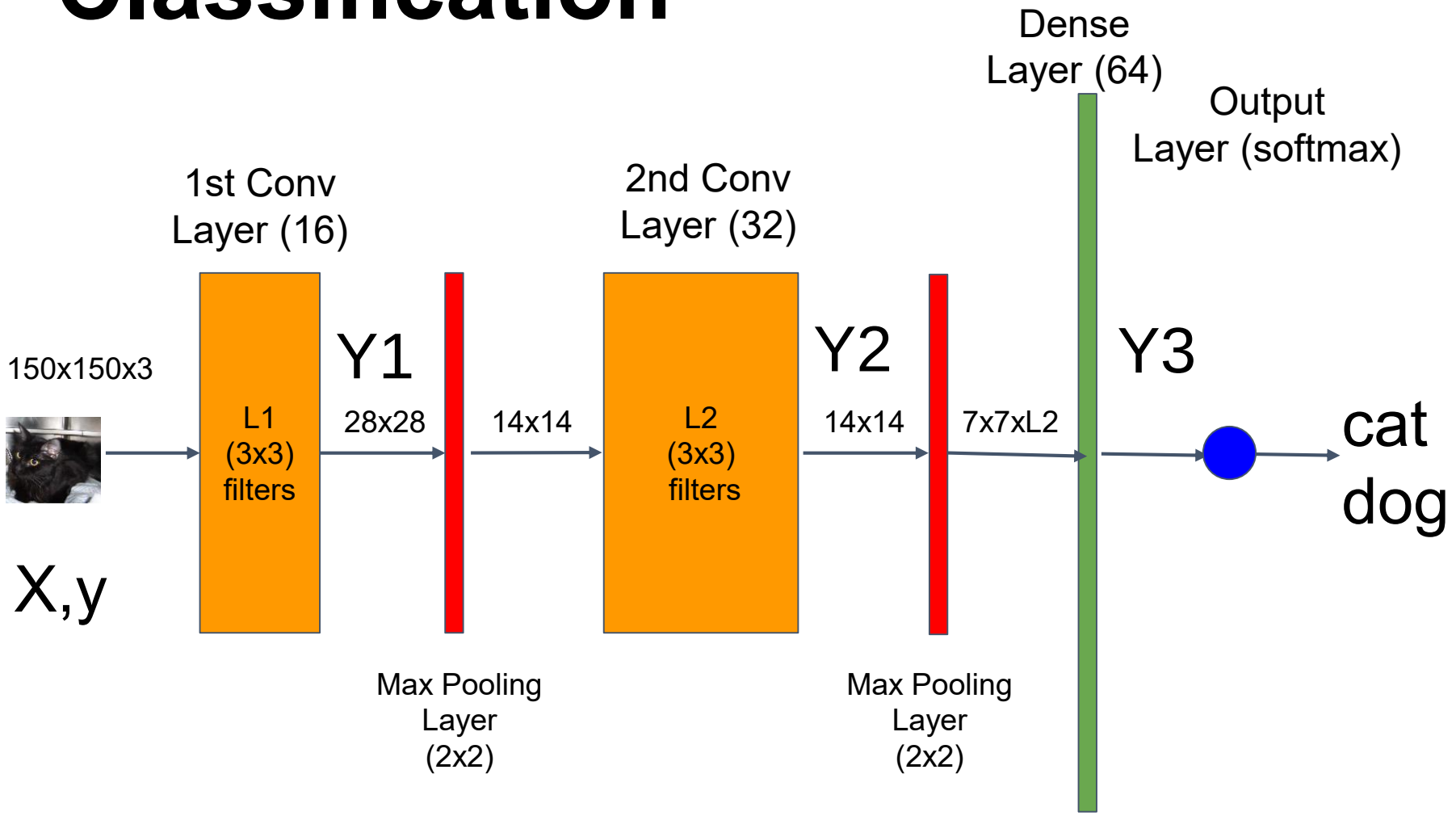
test_dir = '/content/gdrive/My Drive/data/cats-dogs/test/'

validation_dir = '/content/gdrive/My Drive/data/cats-dogs/validation/'





CNN for Cat-Dog Classification



CNN Model for Cats and Dogs

```
model = Sequential()
model.add(Conv2D(16,(3,3),activation='relu',input_shape=(150,
150,3),padding='same'))
model.add(MaxPooling2D(2,2))
model.add(Conv2D(32,(3,3),activation='relu',
                        padding='same'))
model.add(MaxPooling2D(2,2))
model.add(Flatten())
model.add(Dense(64,activation='relu'))
model.add(Dense(1,activation='sigmoid'))
```

Import Images With ImageDataGenerator

```
from keras.preprocessing.image import ImageDataGenerator

train_datagen = ImageDataGenerator(rescale=1./255)
test_datagen = ImageDataGenerator(rescale=1./255)

train_dir = '/content/gdrive/My Drive/data/cats-dogs/train/'
test_dir = '/content/gdrive/My Drive/data/cats-dogs/test/'

train_generator = train_datagen.flow_from_directory(train_dir,target_size=(150,150),
                                                    batch_size=20,class_mode='binary')
test_generator = test_datagen.flow_from_directory(test_dir,target_size=(150,150),
                                                  batch_size=20,class_mode='binary')
```

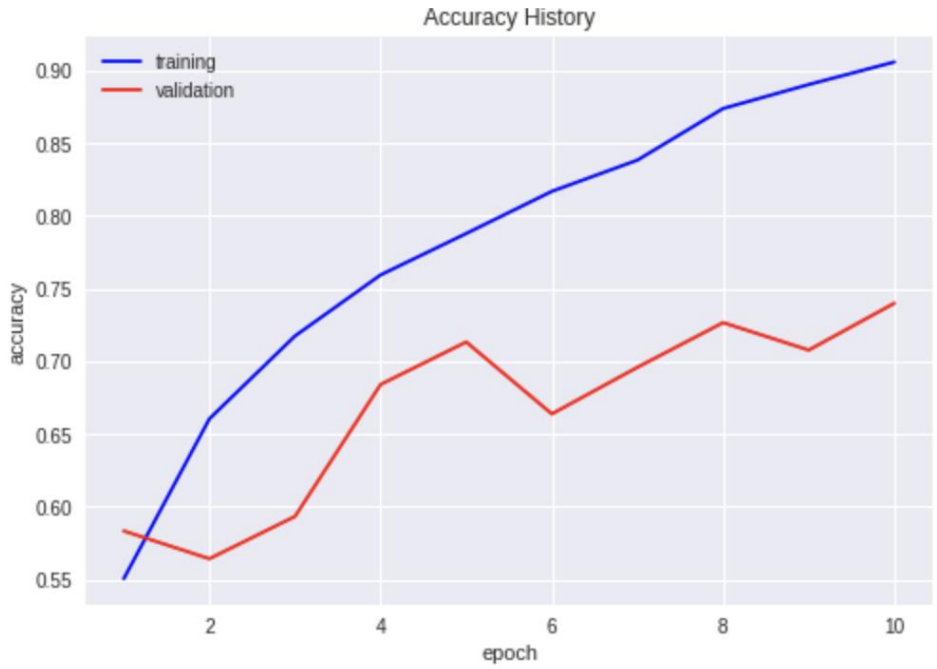
Training with Fit_Generator

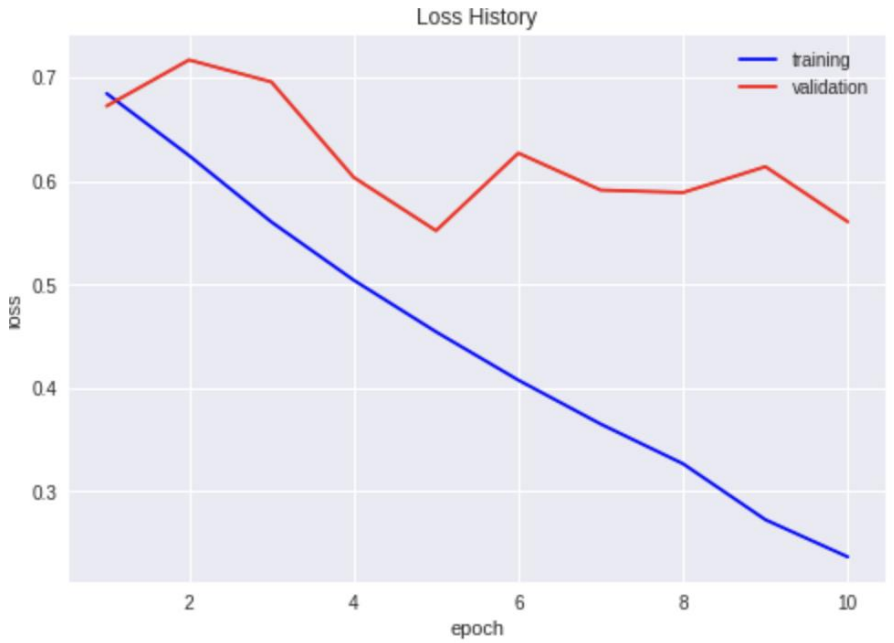
Use Fit Generator method to train the model from folder

```
history =
model.fit_generator(train_generator,epochs=10,steps_per_epoch=100,
                    validation_data=test_generator,validation_steps=100)
```

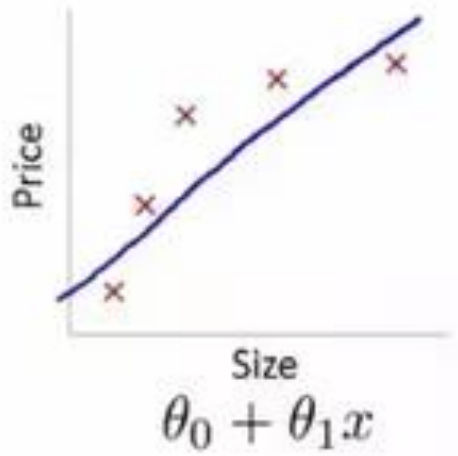
Overfitting Issue

If validation loss increase/saturate with epochs it might indicate overfitting issue



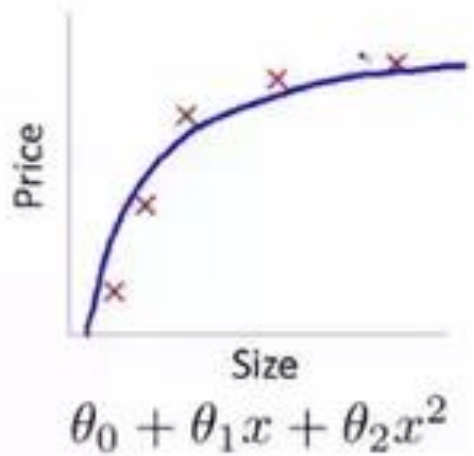


Underfitting vs Overfitting



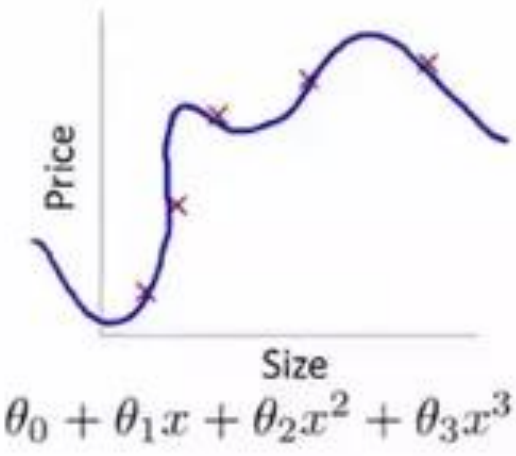
$\theta_0 + \theta_1 x$

High bias
(underfit)



$\theta_0 + \theta_1 x + \theta_2 x^2$

"Just right"



$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

High variance
(overfit)

- Train more epochs

- Train with more data
- Regularization

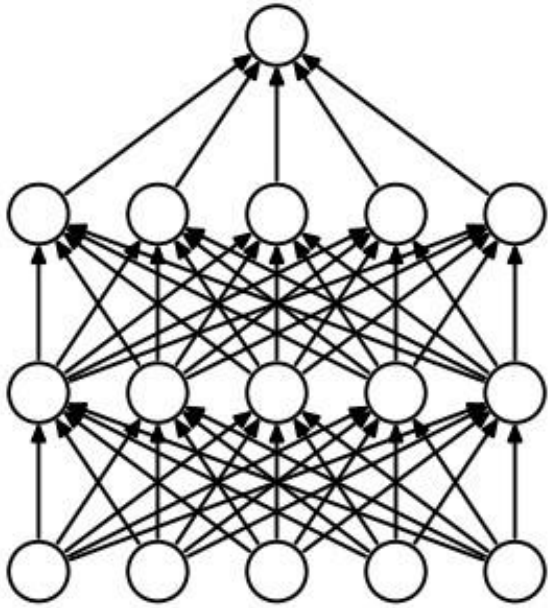
Solving OverFitting Issue

In order to solve overfitting issue, you can try by

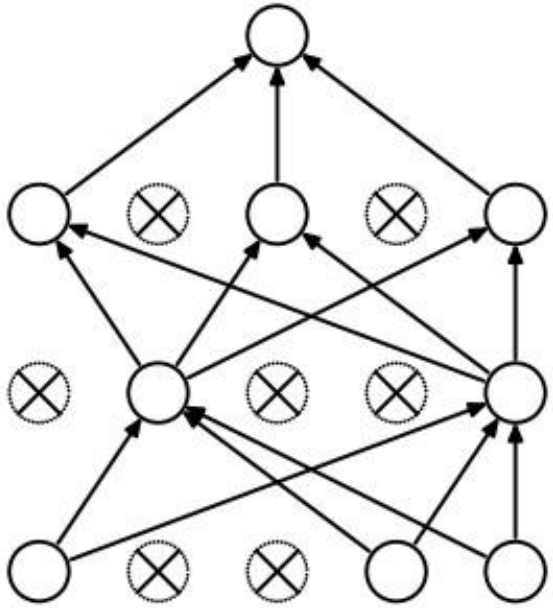
- Data Augmentation
- Dropout

Dropout

- Dropout is a way regularize the parameters, thus reduce over-fitting and general fit better.
- During training, we randomly select activation functions and make the output equal to zero.
- It is generally apply to the fully connected layer.



(a) Standard Neural Net



(b) After applying dropout.

Data Augmentation

New data can be generated from translation, flipping, rotation, shearing, zooming.



Add Dropout

```
model = Sequential()
model.add(Conv2D(16,(3,3),activation='relu',input_shape=(150,150,3),padding='same'))
model.add(MaxPooling2D(2,2))
model.add(Conv2D(32,(3,3),activation='relu',padding='same'))
model.add(MaxPooling2D(2,2))

model.add(Flatten())
model.add(Dense(64,activation='relu'))
model.add(Dropout(0.5))

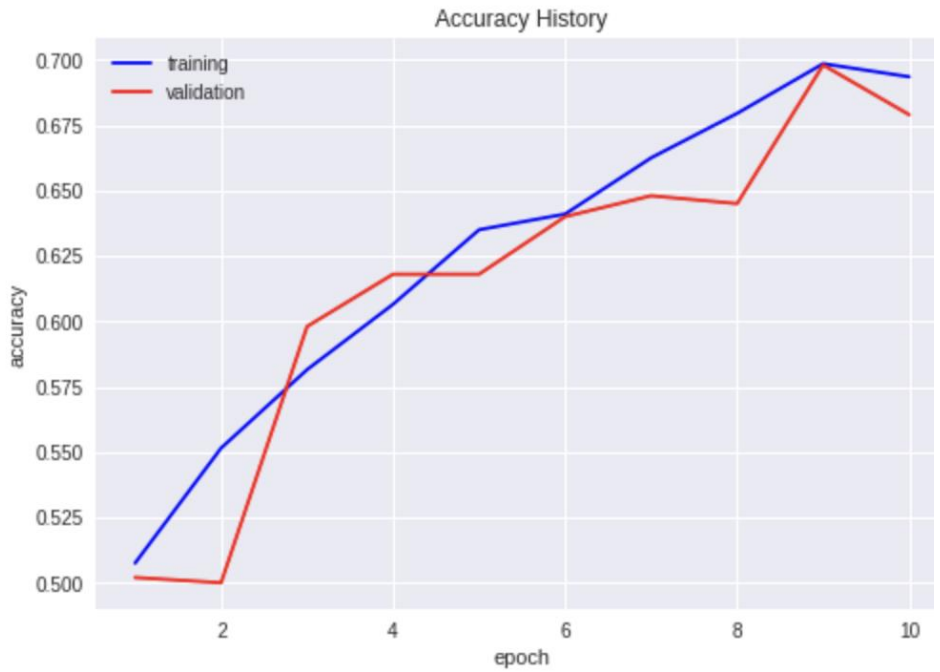
model.add(Dense(1,activation='sigmoid'))
```

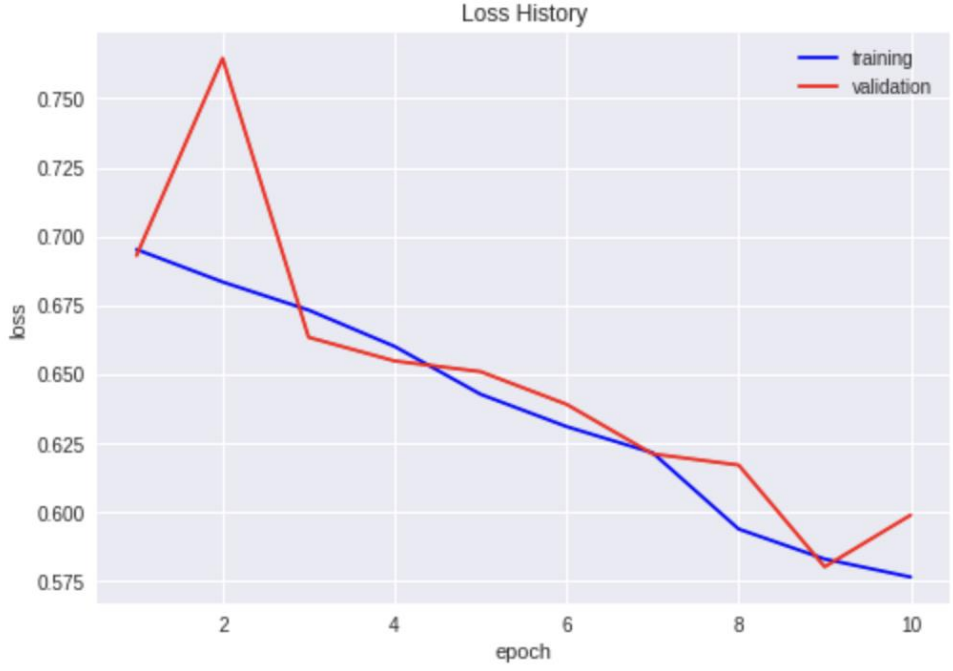
Implement Data Augmentation

```
train_datagen = ImageDataGenerator(rescale=1./255,
                                   rotation_range=40,
                                   width_shift_range=0.2,
                                   height_shift_range=0.2,
                                   shear_range=0.2,
                                   zoom_range=0.2,
                                   horizontal_flip=True
                                   )
test_datagen = ImageDataGenerator(rescale=1./255)
```

Fixing Overfitting with Data Aug

- Using Dropout and Data Augmentation
- Accuracy is ~70% after 10 epochs and can be improved with more epochs

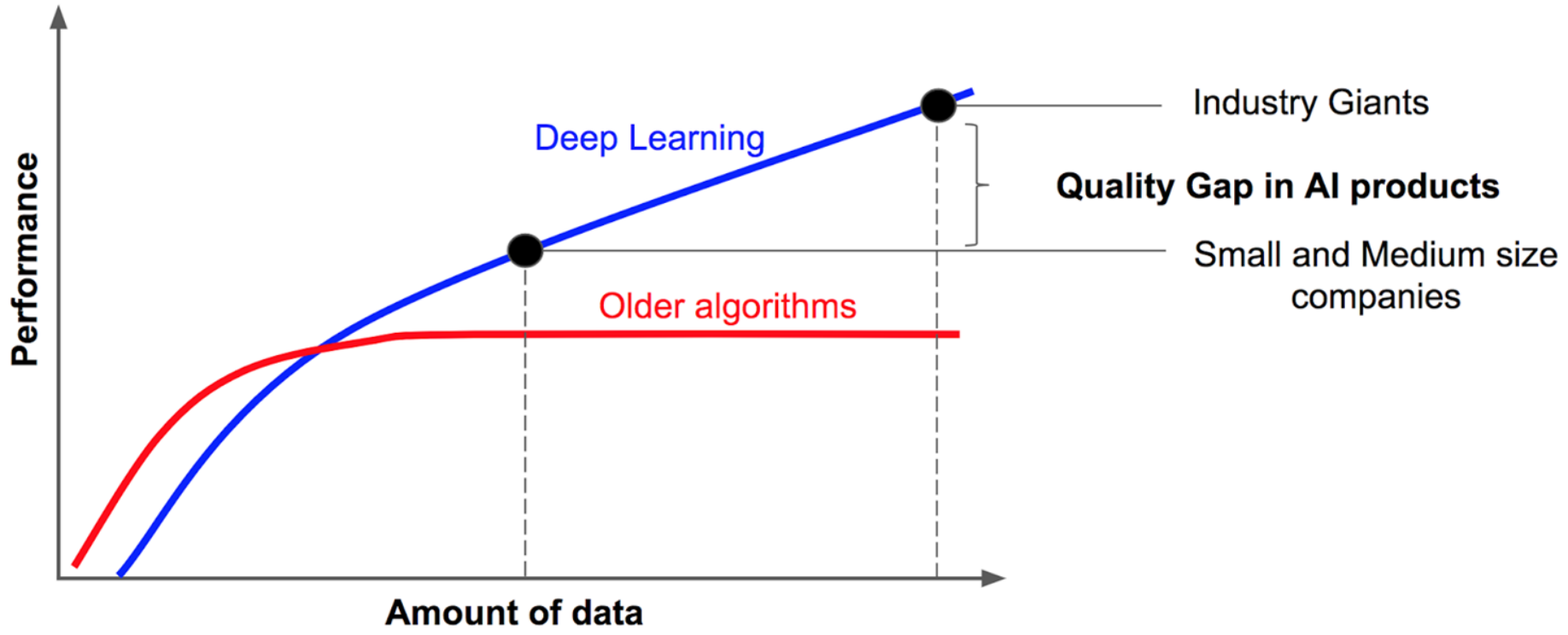




Transfer Learning

Data is the Food for Deep Learning

With more data for training, the performance of Deep Learning can surpasses classical machine learning



Small Training Data Set Issue

Deep NN requires large data set to be a robust model. To overcome small data set issue, people use transfer learning from pre-trained models



Small Data

- Transfer Learning
- More hand-craft tuning
- Reduce layers

Large Data

- Less hand-craft tuning
- Use deeper NN model

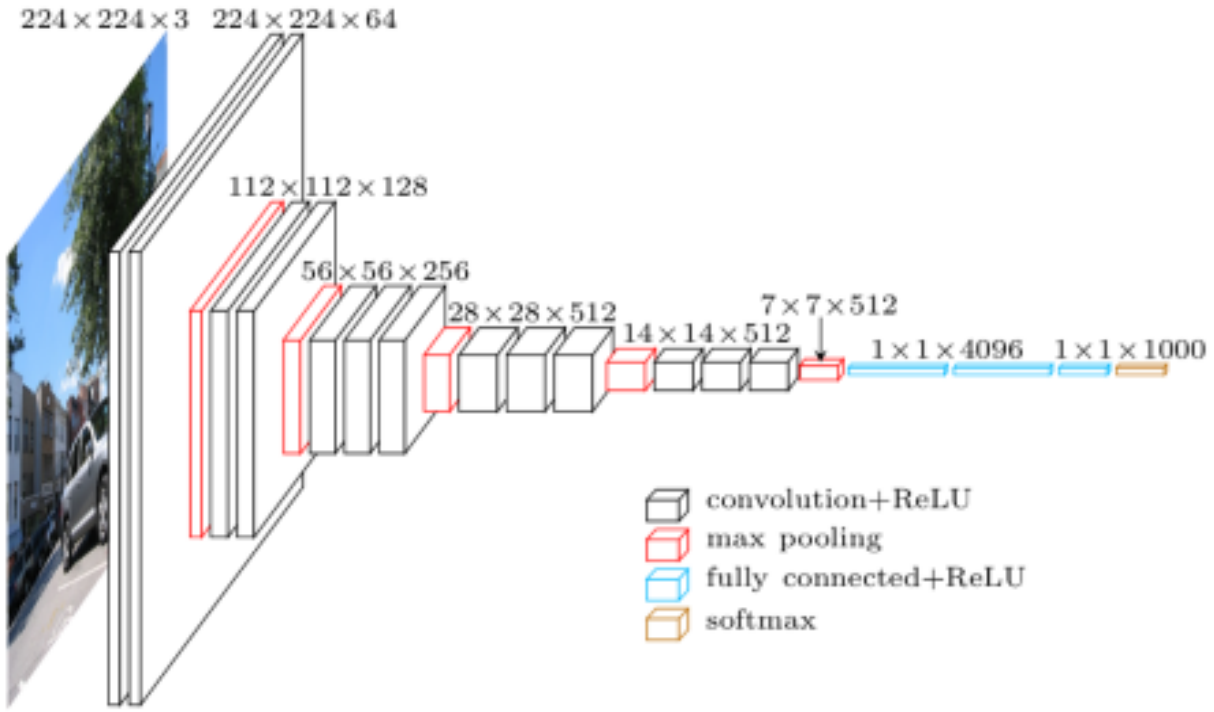
Pre-Trained Models in Keras

Well known trained models include:

- Xception
- VGG16
- VGG19
- ResNet50
- InceptionV3
- InceptionResNetV2
- MobileNet
- DenseNet
- NASNet

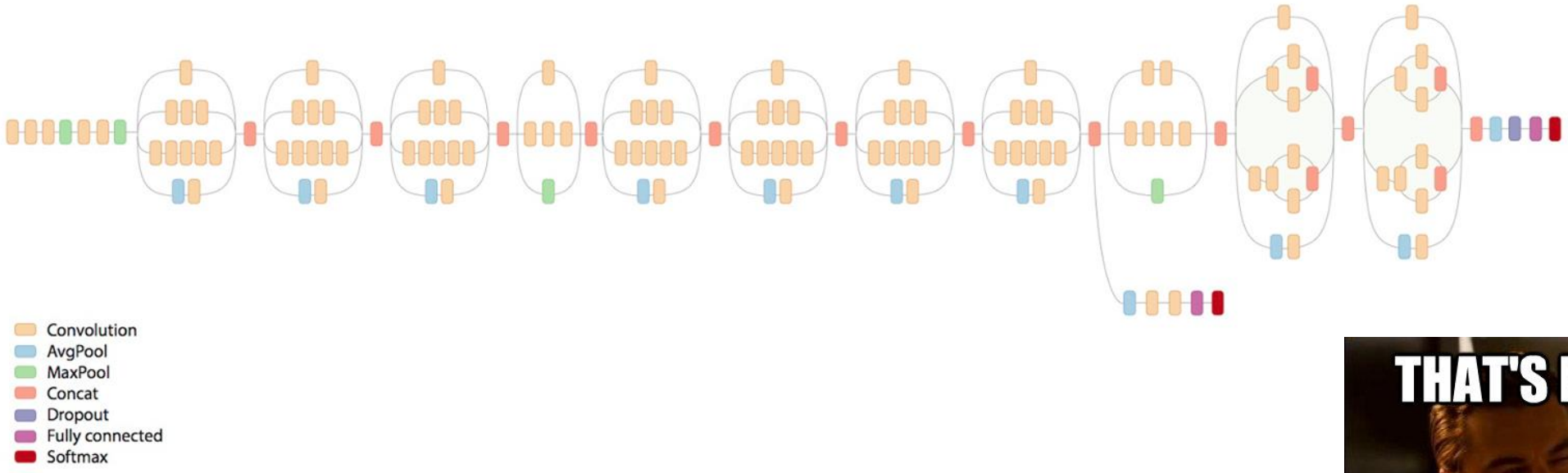
VGG16

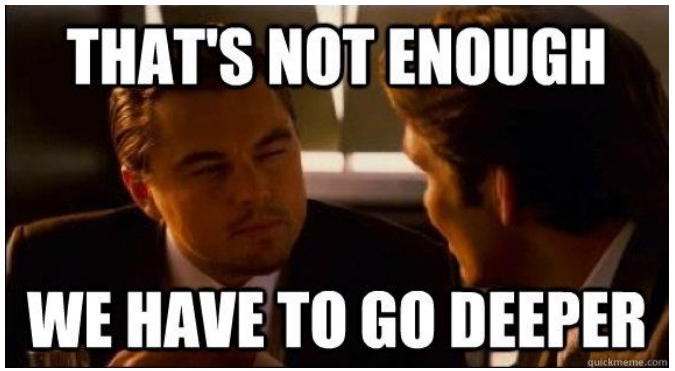
- Pre-trained on ImageNet. Input image size is 224x224px
- Simple architecture - 3x3 filters, 2x2 max pooling, double filter numbers after each max pooling



Inception

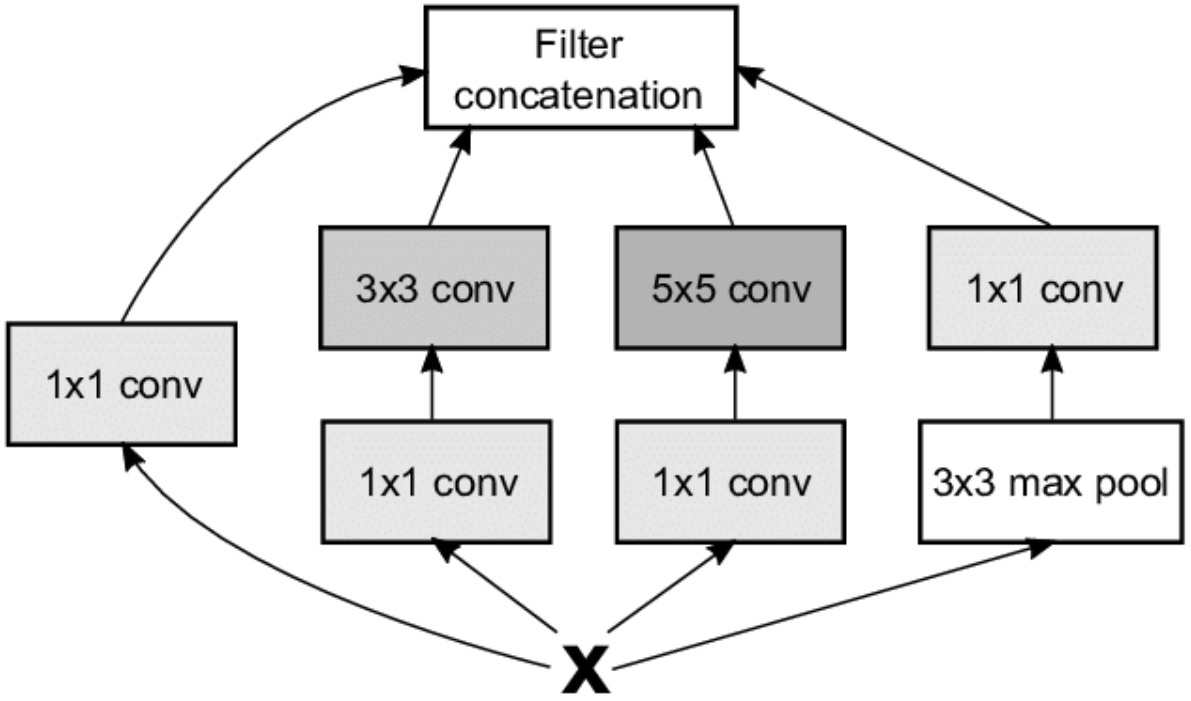
- Winner of ILSVRC 2014, also known as GoogleNet
- Pre-trained on ImageNet. Input Image size is 299x299px
- Consist of many inception modules





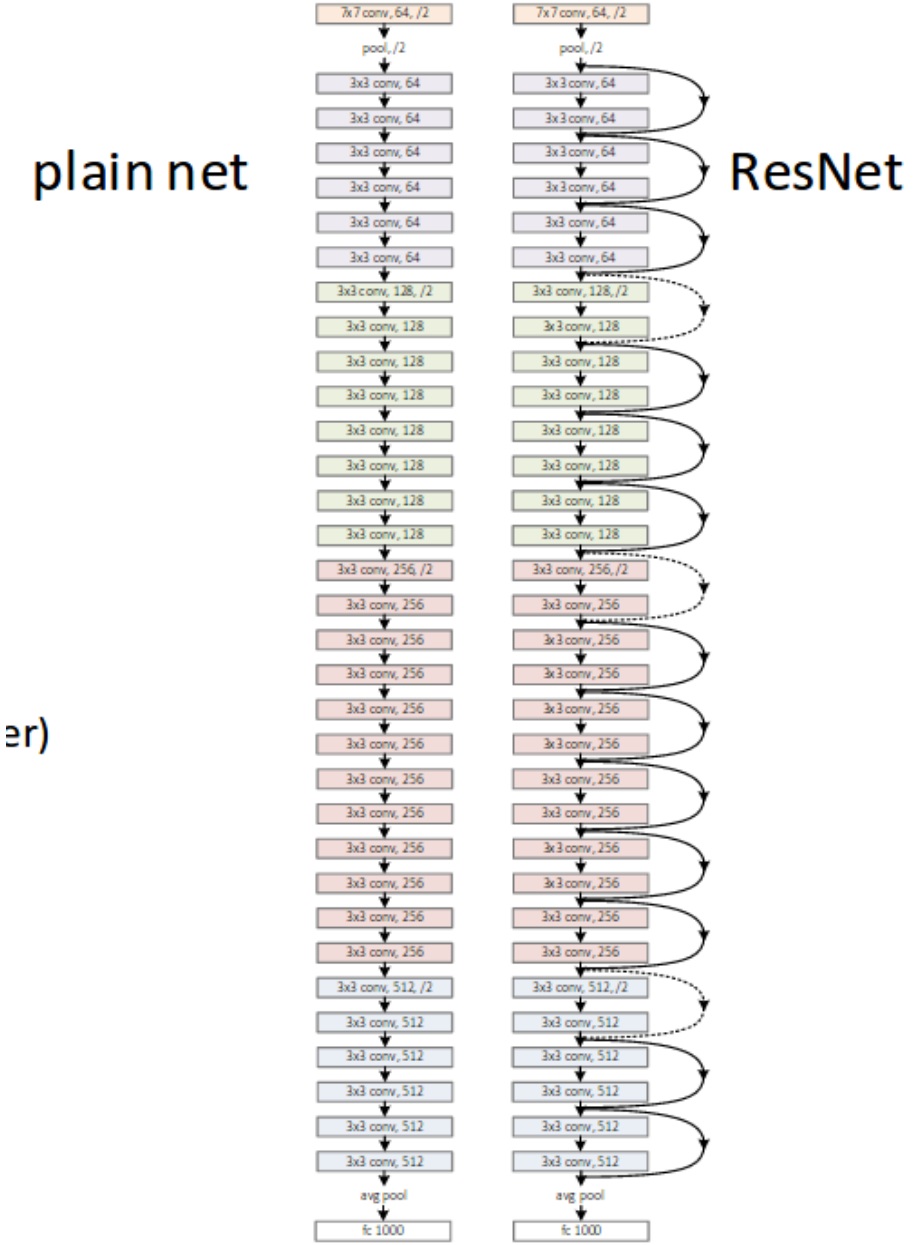
Single Inception Module

The basic idea of inception is to apply different filter sizes eg 1x1, 3x3 and 5x5



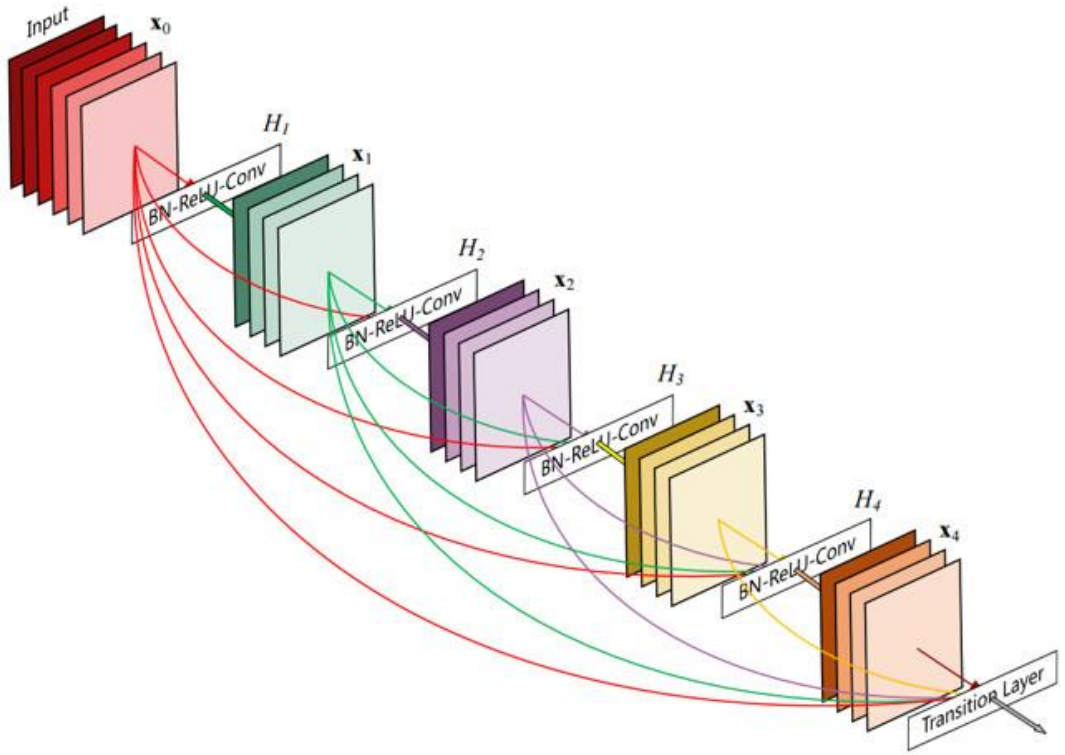
ResNet

- Residual network - winner of ILSVRC 2015 from Microsoft
- Pre-trained on ImageNet. Input image size is 224x224px
- With more layers, the NN performance might degrade due to vanishing gradient issue.
- ResNet allow NN to go deeper by skipping the bottleneck layers



DenseNet

Dense Convolutional Network (DenseNet) is an extension of ResNET, which connects each layer to every other layer in a feed-forward fashion.



VGG16 TF Demo

```
from keras.applications.vgg16 import VGG16)
img = image.load_img('.. pic.jpg',target_size=(224,224))
x = image.img_to_array(img)
x = np.expand_dims(x,axis=0)
predictions = model.predict(x)
classes = decode_predictions(predictions,top=3)
```







Ex: Transfer Learning

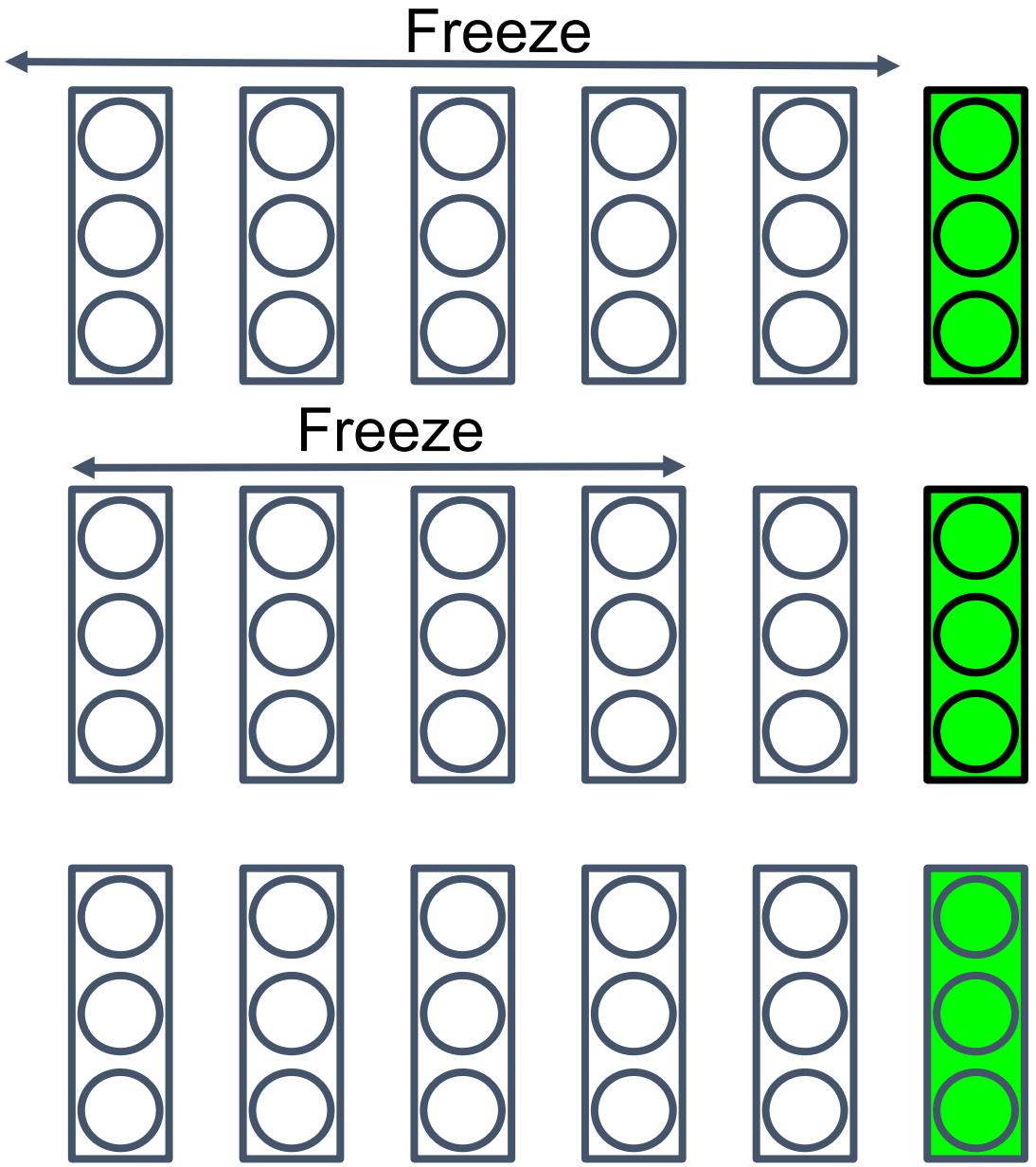
Use Another pretrained model to test

- RESNET
- DenseNET
- NASNET





Fine Tune Pre-Trained Models



Replace the softmax layer
Freeze all layers

Retrain the top few layers and freeze the rest

Retrain the entire layers with preloaded weights as initialization

Loading VGG16 Model

```
from keras.applications.vgg16
import VGG16

base_model =
VGG16(weights='imagenet'
,include_top=False,
input_shape=(150,150,3))
```

include_top to False will remove the dense NN and softmax layers.

```
1 base_model.summary()
```

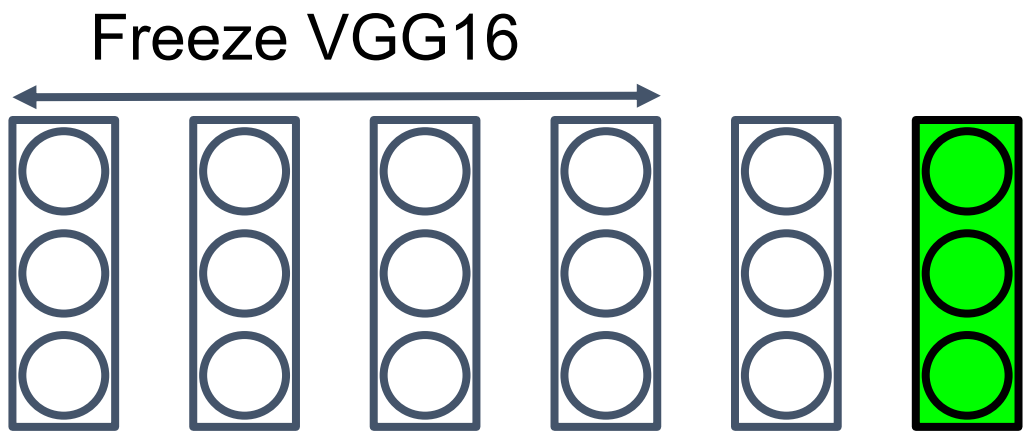
```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
input_1 (InputLayer)         (None, 150, 150, 3)       0
_________________________________________________________________
block1_conv1 (Conv2D)        (None, 150, 150, 64)      1792
_________________________________________________________________
block1_conv2 (Conv2D)        (None, 150, 150, 64)      36928
_________________________________________________________________
block1_pool (MaxPooling2D)   (None, 75, 75, 64)        0
_________________________________________________________________
block2_conv1 (Conv2D)        (None, 75, 75, 128)       73856
_________________________________________________________________
block2_conv2 (Conv2D)        (None, 75, 75, 128)       147584
_________________________________________________________________
block2_pool (MaxPooling2D)   (None, 37, 37, 128)       0
_________________________________________________________________
block3_conv1 (Conv2D)        (None, 37, 37, 256)       295168
_________________________________________________________________
block3_conv2 (Conv2D)        (None, 37, 37, 256)       590080
_________________________________________________________________
block3_conv3 (Conv2D)        (None, 37, 37, 256)       590080
_________________________________________________________________
block3_pool (MaxPooling2D)   (None, 18, 18, 256)       0
_________________________________________________________________
block4_conv1 (Conv2D)        (None, 18, 18, 512)       1180160
_________________________________________________________________
block4_conv2 (Conv2D)        (None, 18, 18, 512)       2359808
_________________________________________________________________
block4_conv3 (Conv2D)        (None, 18, 18, 512)       2359808
_________________________________________________________________
block4_pool (MaxPooling2D)   (None, 9, 9, 512)         0
_________________________________________________________________
block5_conv1 (Conv2D)        (None, 9, 9, 512)         2359808
_________________________________________________________________
block5_conv2 (Conv2D)        (None, 9, 9, 512)         2359808
_________________________________________________________________
block5_conv3 (Conv2D)        (None, 9, 9, 512)         2359808
_________________________________________________________________
block5_pool (MaxPooling2D)   (None, 4, 4, 512)         0
=================================================================
Total params: 14,714,688
Trainable params: 14,714,688
Non-trainable params: 0
_________________________________________________________________
```

Fine Tune VGG16 Model

```
from keras.models import Sequential
from keras.layers import Dense, Flatten

model = Sequential()
model.add(base_model)
model.add(Flatten())
model.add(Dense(1024,activation='relu'))
model.add(Dense(1,activation='sigmoid'))

for layer in base_model.layers:
  layer.trainable = False
```



Training Classification Layers

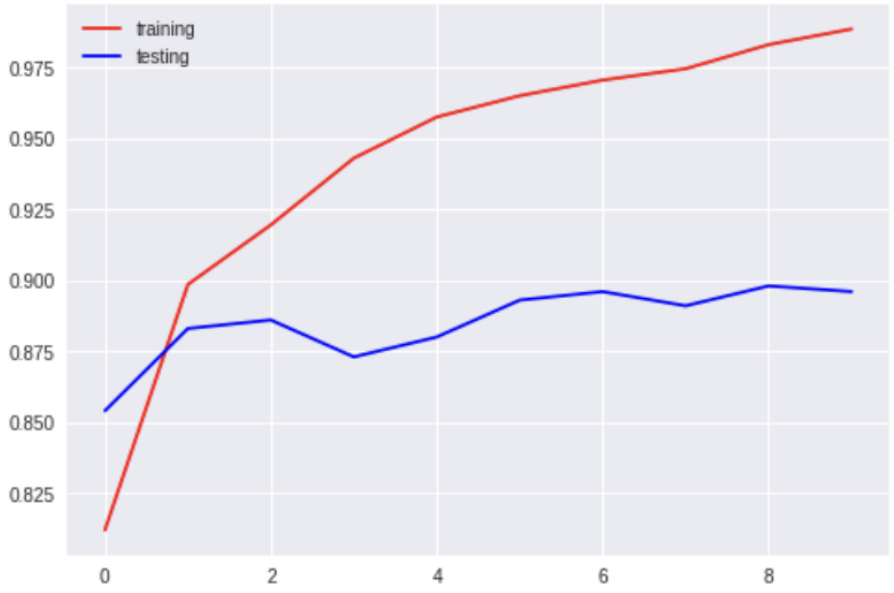
```
# Reduce learning rate
from keras import optimizers
model.compile(optimizer=optimizers.RMSprop(lr=1e-5),
loss='binary_crossentropy',metrics=['acc'])


history =
model.fit_generator(train_generator,epochs=10,steps_per_epo
ch=100,
validation_data=test_generator,validation_steps=100)
```

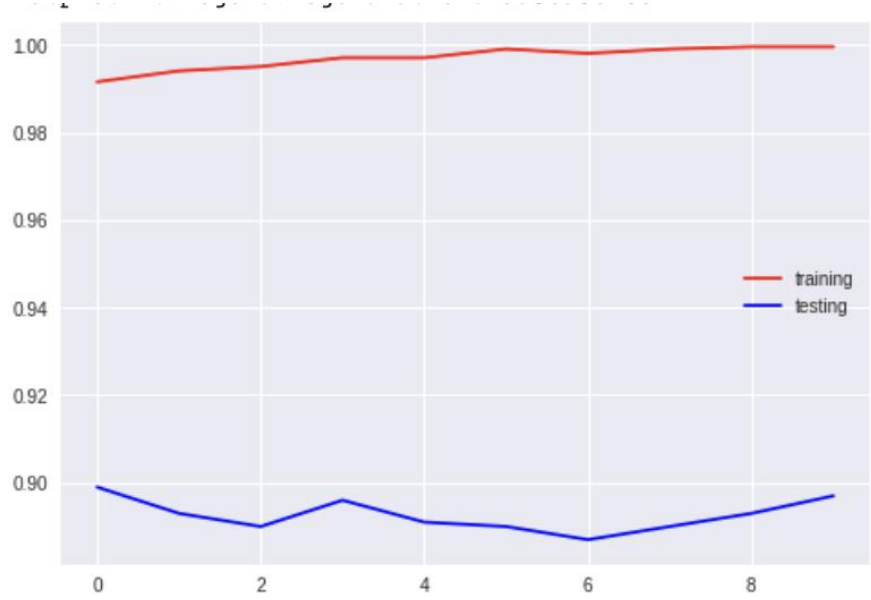
Result

Fine tuning VGG16 has greatly improve the accuracy of the cats and dogs classification to ~90%

Without Data Augmentation



With Data Augmentation



Model Prediction

```
from keras.preprocessing import image

img = image.load_img('/content/gdrive/My Drive/data/cats-dogs/test/dogs/dog.1003.jpg', target_size=(150,150))

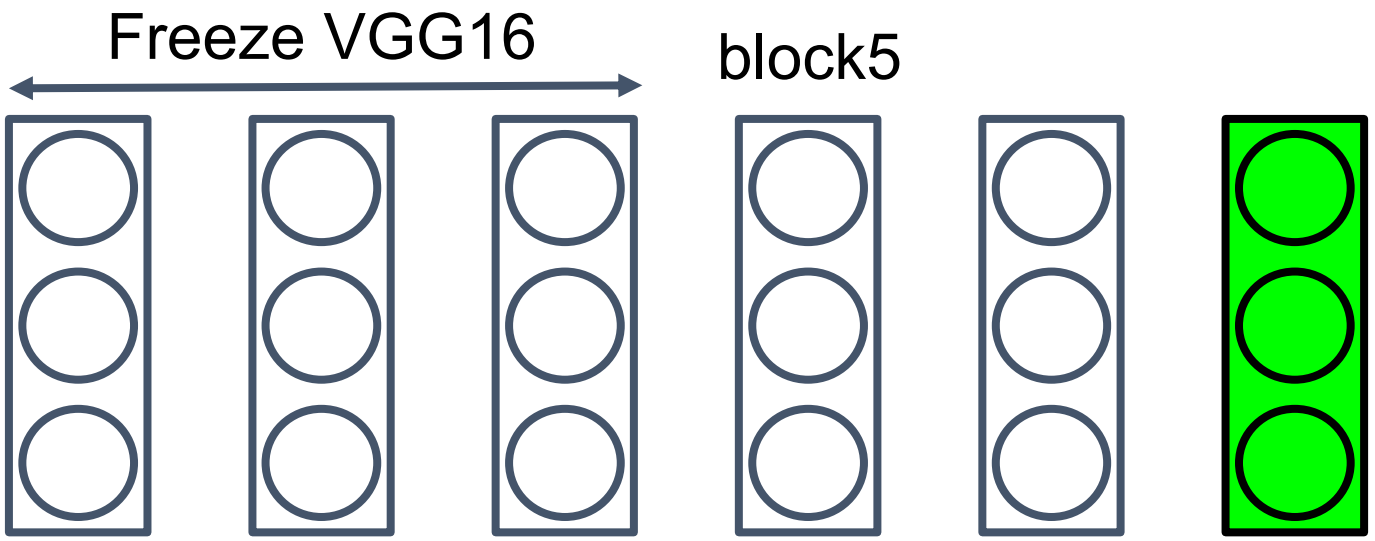

x = image.img_to_array(img)
x = np.expand_dims(x,axis=0)
model.predict(x)
```

Ex: Fine Tuning

Unfreeze block5 layers for fine tuning

Hint:

```
for layer in base_model.layers[-4:]:
  layer.trainable = True
```



Exercise

Train image classification model using:

- Simple CNN
- Transfer Learning

Train image classification model

- Butterfly Image Dataset: https://www.kaggle.com/datasets/phucthaiv02/butterfly-image-classification
- Train model using:
  - Simple CNN
  - Transfer Learning from pre-trained CNN models

Thank you …